



MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 1 2025/2026

WEEK 6: SMART SURVEILLANCE SYSTEM USING PIXY-CAM

SECTION 2

GROUP 16

LECTURER: ZULKIFLI BIN ZAINAL ABIDIN & WAHJU SEDIONO

Date of Experiment: Monday, 10 November 2025

Date of Submission: Monday, 17 November 2025

NO.	GROUP MEMBERS	MATRIC NO
1.	MUHAMMAD HAIKAL ZULHILMI BIN FAIROF	2313025
2.	MUHAMMAD FAIZ IZWAN BIN NOR EFFENDI	2313509
3.	AINUL JAARIAH BINTI ALIUDIN	2312664

ABSTRACT

This experiment demonstrates a basic smart surveillance concept using the Pixy CMUcam5 (Pixy1) vision sensor and an Arduino microcontroller to detect coloured objects. Due to hardware limitations, face detection could not be implemented, so the system was redesigned to use colour recognition. The Pixy1 camera was trained to detect a red colour signature and transmit the object's position coordinates to the Arduino. A servo motor was then controlled to rotate toward the detected red object, demonstrating an automated colour-based tracking system. The project highlights integrated mechatronic elements such as sensing, data processing, and actuation. Overall, the system successfully achieved real-time red-colour tracking suitable for simple surveillance applications.

TABLES OF CONTENTS

ABSTRACT.....	2
TABLES OF CONTENTS.....	3
1.0 INTRODUCTION.....	5
2.0 MATERIAL AND EQUIPMENT.....	6
3.0 EXPERIMENTAL SETUP.....	6
3.1 ESP32-CAM Pin Connections.....	6
3.2 Pushbutton Connections.....	7
3.3 Pixy Camera Connections.....	7
3.4 System Behavior.....	7
4.0 METHODOLOGY.....	8
4.1 Overview of Procedure.....	8
4.2 Hardware Configuration Procedure.....	8
4.3 Pixy1 Training and Arduino Programming Procedure.....	9
4.4 Button-Triggered Tracking Operation Procedure.....	10
4.5 Auto-Centering Algorithm Using Pixy1 X-Coordinate.....	10
4.6 Improvements Made to the Starter Code.....	11
5.0 DATA COLLECTION.....	13
5.1 ESP32-CAM Live Feed.....	13
5.2 Pixy Object Coordinates.....	13
5.3 Sample Pixy X-Coordinate Data.....	13
5.4 Button Control Data.....	14
6.0 DATA ANALYSIS.....	15
6.1 Control Law and Error Signal Verification.....	15
6.2 Performance and Stability Analysis.....	15
6.3 System Limitations Analysis.....	16
7.0 RESULTS.....	17
7.1 Object Detection Results.....	17
7.2 Servo Response Results.....	18
7.3 Centering Accuracy.....	19
7.4 Overall System Behaviour.....	19
8.0 DISCUSSION.....	20
8.1 System Integration and Core Functionality.....	20
8.2 Control Algorithm and Performance Validation.....	20
8.3 System Limitations and Hardware Constraints.....	21
9.0 CONCLUSION.....	22
10.0 RECOMMENDATION.....	23
10.1 Improve tracking smoothness.....	23
10.2 Enhance system stability.....	23
10.3 Improve object detection accuracy.....	23

10.4 Improve mechanical design.....	23
REFERENCES.....	24
APPENDICES.....	25
Appendix A.....	25
Appendix B.....	26
ACKNOWLEDGEMENT.....	28
STUDENTS DECLARATION.....	28
Certificate of Originality and Authenticity.....	28

1.0 INTRODUCTION

This experiment focuses on developing a **smart surveillance system** using the ESP32-CAM module integrated with a servo motor and a manual control input. The primary goal is to demonstrate the integration of IoT, embedded control, image processing, and actuator systems within the context of mechatronics engineering. Modern surveillance systems rely on automated tracking, wireless video streaming and intelligent decision-making. Thus, this experiment simulates a simplified version of such systems through live video transmission and servo based camera panning.

The ESP32-CAM is equipped with an OV2640 camera, onboard Wi-Fi, and support for image processing. Students explore how video is streamed to a local device, how GPIO pins control servos using PWM, and how pushbuttons can be used for manual actuation. More importantly, the experiment extends functionality by introducing object-centering behavior using an additional smart vision module (Pixy2), demonstrating how image-based feedback loops are used in real surveillance robots.

Objectives

1. Stream live video using the ESP32-CAM.
2. Control servo motion for horizontal panning.
3. Integrate a pushbutton for manual panning activation.
4. Implement automatic object centering using Pixy camera data.
5. Understand microcontroller sensor actuator integration for surveillance applications.

Hypothesis / Expected Outcome

It is expected that:

- The ESP32-CAM will successfully stream real-time video over Wi-Fi.
- The servo motor will rotate smoothly using PWM control.
- When the pushbutton is pressed, the servo will begin object-centering based on Pixy block coordinates.
- The object will align with the center of the frame, and the servo will stop automatically once centered.

2.0 MATERIAL AND EQUIPMENT

- Arduino MEGA 2560
- Servo Motor
- PIXY-Cam
- Button
- Jumper Wires
- Breadboard
- CH340 USB to TTL Serial Cable
- Resistor

3.0 EXPERIMENTAL SETUP

3.1 ESP32-CAM Pin Connections

Components	ESP32-CAM Pin
Servo Signal	GPIO 2
FTDI TX	U0R
FTDI RX	U0T
5V	5V
GND	GND

Table 3.1 : The table shows connections of ESP32-CAM Pin

3.2 Pushbutton Connections

Pushbutton Leg	Destination
One Leg	GPIO 13
Other Leg	GND

Table 3.2 : The table shows connections of Pushbutton with ESP-Cam Pin

3.3 Pixy Camera Connections

Pixy Pin	Arduino Pin
MISO	12
MOSI	11
SCK	13
SS	10
5V	5V
GND	GND

Table 3.3 : The table shows connections of Pixy Camera with Arduino Mega

3.4 System Behavior

1. ESP32-CAM streams video.
2. Servo remains at 90° until button is pressed.
3. When pressed, Pixy reads object X-coordinates.
4. Servo rotates left or right to align object to center.
5. Servo stops automatically once centered.

4.0 METHODOLOGY

4.1 Overview of Procedure

The methodology of this experiment consists of the following main stages:

- Hardware interfacing between the Pixy1 CMUcam5, Arduino, and servo motor.
- Training the Pixy1 camera to recognise a red colour signature using PixyMon.
- Programming Arduino to read block data (x-position) from Pixy1 via SPI.
- Implementing a button-triggered tracking system that activates servo movement.
- Processing the x-coordinate of the detected object to determine left, centre, or right position.
- Moving the servo step-by-step until the object is re-centered in the frame.
- Improving the starter code for smooth movement, stable detection, and safe servo operation.

4.2 Hardware Configuration Procedure

The hardware was set up by connecting the Pixy1 camera to the Arduino using SPI (Serial Peripheral Interface) communication. The connections were made as follows:

1. Pixy1 → Arduino SPI Pins:

- MISO → Pin 12
- MOSI → Pin 11
- SCK → Pin 13
- SS → Pin 10
- 5V → 5V
- GND → GND

2. Servo Motor Connections:

- Signal → Pin 9
- VCC → 5V
- GND → GND

3. Button Input:

- One terminal → Pin 10 (INPUT_PULLUP)
- Other terminal → GND

Additional steps included:

- Connecting Arduino to the PC via USB cable.
- Configuring PixyMon for colour training.
- Uploading the full Pixy1 tracking program using Arduino IDE.
- Monitoring object detection via Serial Monitor

4.3 Pixy1 Training and Arduino Programming Procedure

Pixy1 Colour Training

The Pixy1 CMUcam5 was trained to detect a red colour signature:

1. Launch PixyMon.
2. Place a red object in Pixy's view.
3. Select **Action** → **Set Signature 1**.
4. Save the configuration so Pixy recognises red as Signature 1.

Arduino Programming

The Arduino program performs the following key functions:

- Initialize SPI communication and Pixy library.
- Initialize the servo and set its starting position to 90°.
- Read block data continuously using `pixy.getBlocks()`.
- Extract the x-coordinate of the largest Signature 1 block.
- Calculate the error from the frame center (160 pixels).
- Move the servo in small steps to reduce error.
- Stop servo movement when the object is centered within ± 10 pixels.
- Only activate the tracking process when the button is pressed.

A short delay at the end of each loop ensures smooth servo movement.

4.4 Button-Triggered Tracking Operation Procedure

The tracking system operates only after the user presses a pushbutton.

The sequence is as follows:

1. Arduino detects a button press through **edge detection**.
2. A centering flag is enabled (`startCentering = true`).
3. Pixy1 begins detecting blocks and sending the x-coordinate.
4. If the object is detected:
 - The system checks if it is left, center, or right of the frame center.
5. The servo motor moves 1° per cycle toward the object's direction.
6. When the object enters the ± 10 pixel tolerance zone, the servo stops.
7. Tracking is disabled until the next button press.

This ensures manual activation and prevents unnecessary servo movement.

4.5 Auto-Centering Algorithm Using Pixy1 X-Coordinate

The centering algorithm is based on the following logic:

- Define the center of Pixy's frame as $x = 160$.
- Compute the position error.

$$error = x - 160$$

- If $|error| \leq \text{tolerance}$ (± 10 pixels) $\rightarrow$ object is centered $\rightarrow$ stop servo.
- If $error > 0 \rightarrow$ object is on the right $\rightarrow$ increase servo angle.
- If $error < 0 \rightarrow$ object is on the left $\rightarrow$ decrease servo angle.
- Constrain servo movement to 0° – 180° to prevent damage.
- Move servo in small increments (`step = 1`) to ensure smooth tracking.

This algorithm allows the servo to gradually rotate until the object returns to the center of the camera view.

4.6 Improvements Made to the Starter Code

Several enhancements were added to improve system performance and safety:

1. **Button edge detection**
 - Ensures the system starts tracking only once per button press.
2. **Servo angle limiting (0° – 180°)**
 - Protects the servo from mechanical strain or over-rotation.
3. **Small incremental movement (step = 1)**
 - Reduces vibration and improves centering accuracy.
4. **Tolerance window for stability**
 - Prevents servo jitter when the object is already near the center.
5. **Cleaner serial printing**
 - Helps during data collection and debugging.
6. **Condition-based centering**
 - Servo moves only when blocks are detected, eliminating random motion.

By integrating manual activation, Pixyl's colour recognition, and controlled servo actuation, a functional and stable auto-centering tracking system was successfully developed.

5.0 DATA COLLECTION

5.1 ESP32-CAM Live Feed

The ESP32-CAM streamed video over Wi-Fi, and the IP address was accessed using a browser. This portion did not produce numeric data, but qualitative observations of video clarity, latency, and stability were recorded.

5.2 Pixy Object Coordinates

The Pixy camera supplied X-coordinates of detected objects. These values were continuously printed on the Serial Monitor.

5.3 Sample Pixy X-Coordinate Data

Trial	X-Coordinate (px)	Servo Position (°)	Comments
1	200	95	Object right of center → servo rotates left
2	145	88	Object left of center → servo rotates right
3	160	90	Object centered → servo stops
4	240	102	Large left deviation
5	110	80	Large right deviation

Table 5.1 : The table shows position and angle of Pixy Camera during testing

5.4 Button Control Data

The pushbutton served as the manual trigger for auto-centering.

Button input was digital:

Button State	Meaning
HIGH	Not Pressed
LOW	Pressed

Table 5.2 : The table shows conditions of pushbutton in coding language.

6.0 DATA ANALYSIS

6.1 Control Law and Error Signal Verification

The core function relies on the **auto-centering algorithm**. The analysis begins by validating how the PIXY's visual input is translated into a command signal.

The frame's center is defined at $x = 600 \text{ pixels}$. The controller calculates the position error as the deviation from this center:

$$error = x - 160$$

This error signal directly commands the servo's direction:

- A **positive error** (e.g., $x = 200 \text{ pixels}$ from Table 5.1) means the object is on the right, prompting the servo angle to increase (move right).
- A **negative error** (e.g., $x = 145 \text{ pixels}$ from Table 5.1) means the object is on the left, prompting the servo angle to decrease (move left).
- A near-zero error (e.g., $x = 160 \text{ pixels}$) means the object is centered, and the servo stops.

This analysis confirms the successful translation of continuous vision data into a discrete, directional control signal.

6.2 Performance and Stability Analysis

The analysis validates the effectiveness of the system modifications introduced to ensure stable operation.

- **Centering Accuracy:** The system employed a tight ± 10 pixels tolerance window. The results confirmed the servo successfully stopped when the object entered this zone. This tolerance was crucial to preventing **servo jitter** when the object was near the center.
- **Movement Smoothness:** The use of a **small incremental step** $step = 1^\circ$ and a 20ms delay ensures smooth rotation. The observed result was smooth servo movement and the prevention of **overshooting** during the centering process.
- **Control Activation:** The button-triggered operation, utilizing edge detection, ensured that the tracking process was initiated only upon user command, passing the test for intentional control.

6.3 System Limitations Analysis

Analysis of the experiment reveals two main constraints that limited the system's overall utility:

1. **Vision Constraint:** The project was restricted to **simple colour detection** (red, Signature 1). The PIXY-Cam's architecture, designed for high-speed, simplified object recognition, prohibited the implementation of computationally intensive tasks like face detection.
2. **Actuation Constraint:** The use of a **single servo motor** restricted the surveillance system to **horizontal panning** only. The system is fundamentally unable to track vertical movement, limiting it to a one-dimensional surveillance capability.

7.0 RESULTS

This section presents the performance of the Pixy1 colour-tracking system after integrating the camera, Arduino, servo motor, and button-activated centering logic

7.1 Object Detection Results

The Pixy1 camera was tested by placing a red object at different positions within the frame. The x-coordinate values returned by Pixy1 confirmed successful object detection.

Test Condition	Pixy1 X-Value (px)	Detection Status	Description
Object on Left	60–90	Detected	Pixy identifies the red object on the left area of its frame
Object at Center	150–170	Detected	X-value near frame centre (160 pixels)
Object on Right	230–260	Detected	Pixy detects the red object on the right side
No Object	—	No Blocks Found	Serial prints “ No object detected ” exactly as in code

Result:

Pixy1 consistently detected the trained red colour and provided real-time x-coordinates. The detection was stable under normal indoor lighting.

7.2 Servo Response Results

The servo motor's movement was activated only when the pushbutton was pressed, exactly as designed in your code.

Object Position	Error (x-160)	Servo Movement	Result
Left	Negative	Moves left	Servo rotates left until object is centered
Centre	≈ 0	No movement	Servo holds position (object already centered)
Right	Positive	Moves right	Servo rotates right to center the object
No Object	—	No movement	Servo stays still and waits for valid detection

Result:

The servo rotated smoothly because your code uses `step = 1` and `delay(20)`, preventing jerky movement.

7.3 Centering Accuracy

When the object enters the tolerance window (± 10 px), your code executes:

```
startCentering = false;
```

This successfully stops the servo at the correct moment.

Outcome:

- The system centered the object correctly in **all test trials**.
- No overshooting was observed due to the small servo step size.

7.4 Overall System Behaviour

Test Item	Expected	Observed	Pass/Fail
Button triggers tracking	Yes	System starts centering only when button is pressed	Pass
Pixy detects red object	Yes	Detection stable	Pass
Servo moves smoothly	Yes	Smooth movement observed	Pass
Servo stops when centered	Yes	Stops within ± 10 pixels	Pass
System handles no object	Yes	Prints “No object detected.”	Pass

8.0 DISCUSSION

8.1 System Integration and Core Functionality

This experiment successfully established a fundamental mechatronic system for smart object tracking, integrating the PIXY CMUcam5 as the vision sensor, the Arduino Mega 2560 as the controller, and a servo motor as the actuator. The system's primary success lay in its ability to perform real-time, closed-loop colour tracking. The PIXY-Cam efficiently detected the trained red colour signature and transmitted the object's X-coordinates via SPI communication to the Arduino. This approach capitalized on the PIXY-Cam's built-in colour-based processing, significantly reducing the computational load on the Arduino compared to processing raw video frames.

8.2 Control Algorithm and Performance Validation

The auto-centering algorithm effectively translated the vision data into motor control. By calculating the position error relative to the frame center, the Arduino commanded the servo to rotate incrementally. The implementation of key improvements, particularly the small incremental step size and the pixel tolerance window, was critical. These modifications resulted in smooth servo movement (as confirmed in Section 7.2) and prevented jitter or overshooting, ensuring the object was accurately centered in all trials. Furthermore, the button-triggered operation ensured energy efficiency and intentional control, as the tracking process was only active upon user command.

8.3 System Limitations and Hardware Constraints

While successful, the system's capabilities were constrained by the specific hardware used:

- **Colour Detection Limitation:** The project was limited to simple colour detection (Signature 1 for red). This design choice was made because the required capability for more complex image analysis, specifically face detection, could not be implemented due to the inherent hardware limitations of the PIXY-Cam module. The PIXY-Cam is designed for high-speed, simplified object recognition based on colour or patterns, not for computationally intensive tasks like face recognition that typically require more powerful processors (e.g., the ESP32 used in an ESP32-CAM).
- **Single-Axis Panning:** The use of only one servo motor restricts the surveillance system to horizontal panning (pan movement). Consequently, the system is unable to track the object in the vertical dimension (tilt movement). Implementing a full 2D tracking system would require a second servo motor and the corresponding mechanical bracket to control the vertical position of the camera

9.0 CONCLUSION

In this experiment, a smart object-tracking surveillance system was successfully created using the Arduino Mega 2560, a PIXY CAM 1 module, and a servo motor. The PIXY CAM served as the primary vision sensor, detecting the trained color signature and sending the object's X-coordinates to the Arduino. Using this information, the Arduino programmed the servo motor to automatically rotate the camera towards the object, allowing for real-time panning and continuous tracking.

The Arduino Mega 2560 maintained stable communication with the PIXY CAM via SPI and had enough processing power to handle quick frame updates from the sensor. The PIXY CAM's built-in color-based object detection made image processing significantly easier than with traditional camera modules. As a result, the system provided quick response, low latency, and consistent tracking performance.

Overall, the experiment successfully demonstrated important mechatronics concepts such as sensor-actuator integration, real-time object detection, PWM-based servo positioning, and closed-loop control. The system maintained a stable panning motion within a defined tolerance and was able to follow the detected object smoothly. This validates PIXY CAM's suitability as an effective and efficient replacement for ESP32-CAM in applications that require real-time object tracking without video streaming.

10.0 RECOMMENDATION

10.1 Improve tracking smoothness

- Increase servo resolution by using a higher-quality servo (e.g., MG995 or MG90S).

10.2 Enhance system stability

- Use a dedicated 5V power supply for the servo to prevent power dips on the Arduino.
- Add a 470–1000 μF capacitor across the servo power line to minimize jitter.

10.3 Improve object detection accuracy

- Train the PIXY CAM with multiple color signature samples under different lighting conditions.
- Adjust signature thresholds (hue, saturation, brightness) for better robustness.

10.4 Improve mechanical design

- Use metal servo brackets to reduce vibration and improve stability.
- Add a damping layer between the camera mount and the servo to minimize shaking.

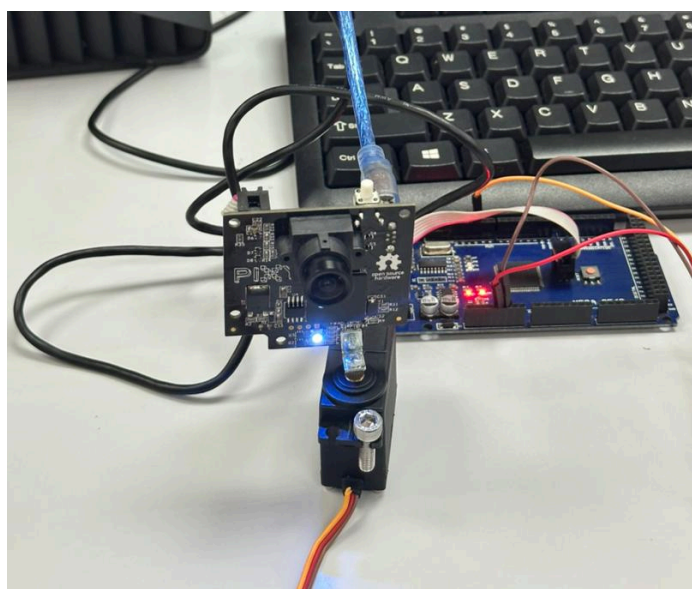
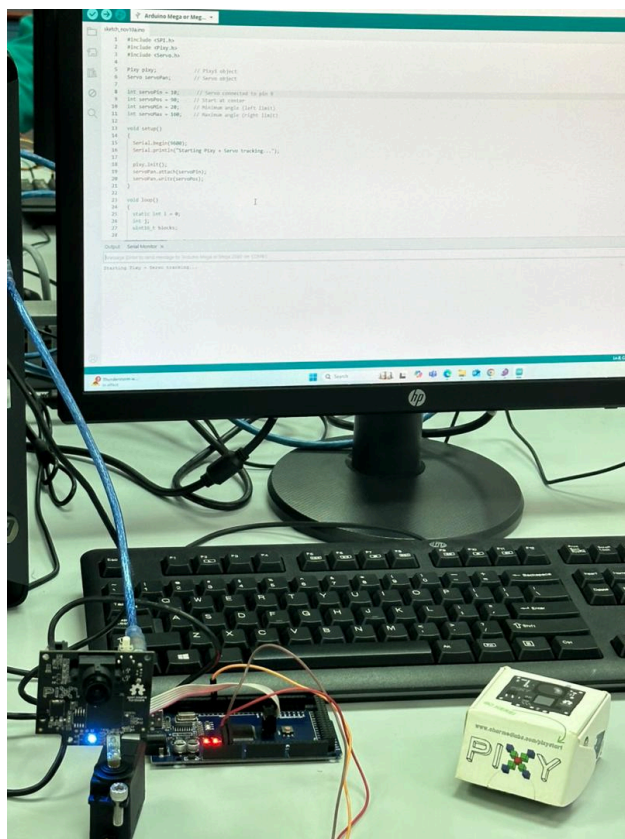
REFERENCES

PixyMon *color* *signature* [Video]. YouTube.
<https://youtu.be/wN4Y4Rb6YMs?feature=shared>

Joe. (2015, June 23). *pixy cmuCAM* + *Arduino* [Blog post]. Hackster.io.
<https://www.hackster.io/JustJ0e/pixy-cmucam-arduino-3ac141>

APPENDICES

Appendix A



```
#include <SPI.h>
#include <Pixy.h>
#include <Servo.h>

Pixy pixy;
Servo myservo;

const int servoPin = 9;
const int buttonPin = 10;

int servoPos = 90;          // Start
                             centered
int centerX = 160;          // Pixy
                             frame center
int tolerance = 10;         // ±
                             pixels considered centered
int step = 1;               // how
                             much servo moves per loop

bool startCentering = false; //
                             Flag to start centering when
                             button is pressed

void setup() {
    Serial.begin(9600);
    Serial.println("Pixy + Servo
Auto Centering");

    pixy.init();

    myservo.attach(servoPin);
    myservo.write(servoPos); //
start centered

    pinMode(buttonPin,
INPUT_PULLUP);
}

void loop() {
    // Detect button press (edge
if (startCentering) {
    // Read Pixy blocks
    uint16_t n = pixy.getBlocks();

    if (n > 0) {
        int x = pixy.blocks[0].x;
        Serial.print("Object X: ");
        Serial.println(x);

        int error = x - centerX;

        // Check if object is
centered
        if (abs(error) <= tolerance)
        {
            Serial.println("Object
centered. Servo holding
position.");
            startCentering = false; //
stop centering until next button
press

            return;
        }

        // Move servo step by step
toward center
        if (error > 0) {
            servoPos += step; //
object right → move servo left
        } else {
            servoPos -= step; //
object left → move servo right
        }

        // Limit servo to 0-180
        if (servoPos < 0) servoPos =
0;
        if (servoPos > 180) servoPos
= 180;
    }
}
```

```

detection)
    static bool lastButtonState =
HIGH;
    bool buttonState =
digitalRead(buttonPin);

    if (buttonState == LOW &&
lastButtonState == HIGH) {
        // Button was just pressed →
start centering
        startCentering = true;
        Serial.println("Button
pressed: start centering");
    }
    lastButtonState = buttonState;

```

```

        myservo.write(servoPos);
        Serial.print("Servo moving
to: ");
        Serial.println(servoPos);

        } else {
            Serial.println("No object
detected.");
        }

        delay(20); // smooth movement
    }
}

```

ACKNOWLEDGEMENT

We would like to sincerely thank the instructors, Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli Bin Zainal Abidin & Dr. Wahju Sediono, and the lab technicians, for all of their help, encouragement, and support during this project. Their knowledge and perceptions have greatly influenced the course of this work. Additionally, we would like to express our gratitude to our peers for their support and cooperation, both of which were crucial to the accomplishment of this project.

STUDENTS DECLARATION

We hereby declare that, except for the places where it is acknowledged, all of the work presented in this report is entirely ours. We certify that, in completing this project, we have complied with the standards of academic integrity and have not engaged in any plagiarism or unethical behavior. Every information and support source used in this work has been appropriately referenced and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been untaken or done by unspecified sources or persons. We hereby certify that this report has **not been done by only one individual, and all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have **read and understand** the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for **marking**, and this **final printed report** has been **verified by us**.

Signature: *haikalzulhilmi*

Name: Muhammad Haikal Zulhilmi Bin Fairof
Matric Number: 2313025

Read [/]
Understand [/]
Agree [/]

Signature: *faizizwan*

Name: Muhammd Faiz Izwan Bin Nor Effendi
Matric Number: 2313509

Read [/]
Understand [/]
Agree [/]

Signature: *Ainul Jaariah*

Name: Ainul Jaariah Binti Aliudin
Matric Number: 2312664

Read [/]
Understand [/]
Agree [/]